Noise-whitened VarPro fitting: combining smooth and discrete basis functions without mass matrices in the fit

N. Alger, Claude (notes; session 2026-07-24)

Abstract

Per-row VarPro fitting of the LG-PSF Hessian model combines two kinds of basis function: theta-dependent smooth (Laguerre–Gaussian) features, and a fixed theta-independent "extra" basis (a discrete spike, and generalizations of it) supplied directly as sampled values. The two live in different spaces with different natural inner products (one in the discretized function space, one in its dual), and combining them naively – e.g. orthogonalizing them against each other under a plain Euclidean inner product – either gets the wrong answer or requires the fitting code to know about the mesh mass matrices explicitly. This note derives a *noise-whitening* change of variables under which the entire per-row fit becomes an ordinary (mass-free) least-squares problem: every basis function, every derivative, and the data itself are rescaled once, at the interface, so that plain Euclidean operations throughout the fit are automatically correct. The two mass matrices (M_1 for the row/target space, M_2 for the column/source space) appear only in that one whitening layer; the VarPro fitting machinery itself never sees them. The whitening transform is theta-independent and symmetric, so it composes with the existing forward/reverse-mode derivative machinery (`lg_functions.hpp`, `ellipsoid_transform.hpp`) with no new derivative math required.

1 Setup

Recall the discretization picture (session 2026-07-23): X is the discrete source/column function space with inner product $\langle u, v\rangle_X = u^\top M_2 v$; Y is the discrete target/row space with $\langle u, v\rangle_Y = u^\top M_1 v$; X', Y' are their duals, with $M_2 : X \to X'$ and $M_1 : Y \to Y'$ the Riesz maps induced by those inner products; the kernel $\Phi : X' \to Y$ and the assembled operator $H = M_1 \Phi M_2 : X \to Y'$. Both M_1, M_2 are diagonal (lumped mass).

Each Laguerre–Gaussian smooth feature $\phi_i(\cdot;\theta)$, sampled at mesh nodes, is an element of X – and, because the continuum family $\{\psi_i\}$ is exactly $L^2(\mathbb{R}^N)$-orthonormal (verified numerically in `lg_harmonics_table.hpp` and `lg_functions.hpp`), the sampled ϕ_i are nearly M_2-orthonormal: $\phi_i^\top M_2 \phi_{i'} \approx \delta_{ii'}$, up to mesh resolution. A user-supplied "extra" basis function e_d (a one-hot vector for the diagonal spike; more general indicator combinations for e.g. a ring-neighbor correction) is instead a sampled *functional* – the coordinate representation of a discrete measure (a sum of point masses) via the FEM nodal interpolation property $N_k(x_i) = \delta_{ik}$ – and so is native to X', not X.

2 The row model

Fix a row ρ with mass $m_\rho := (M_1)_{\rho\rho}$, and a local neighbor/support batch of K column points x_j with per-point masses $m_j := (M_2)_{jj}$. The row model is

$$H[\rho, j] \;\approx\; \underbrace{m_\rho m_j \sum_i c_i\, \phi_i(x_j;\theta)}_{\text{smooth}} \;+\; \underbrace{m_\rho \sum_d s_d\, e_d(j)}_{\text{extra}}, \tag{1}$$

with c_i, s_d the (linear) coefficients VarPro solves for at each trial θ. The column mass m_j on the smooth term is a quadrature weight – ϕ_i approximates a continuum kernel evaluated at a mesh quadrature point, and m_j discretizes the integral. The extra term carries no quadrature weight because it is not a continuum-kernel evaluation at all: it is a direct, discrete correction to specific matrix entries. (Session 2026-07-23 verified this asymmetry reproduces the established $m_\rho s_\rho\, \delta_{\rho j}$ spike convention exactly.)

Probing: draw i.i.d. standard normal z_ℓ ($\ell = 1, \ldots, k$, restricted to the batch – a restriction of an i.i.d. vector is still i.i.d.), observe $y_\ell(\rho) = (H z_\ell)(\rho) = \sum_j H[\rho, j]\, z_\ell(j)$. Substituting (1),

$$y_\ell(\rho) \;=\; m_\rho \sum_i c_i\, g_\ell^{(i)} \;+\; m_\rho \sum_d s_d\, h_\ell^{(d)}, \qquad g_\ell^{(i)} := (M_2 \phi_i)\cdot z_\ell, \quad h_\ell^{(d)} := e_d \cdot z_\ell. \tag{2}$$

The problem with combining these naively. ϕ_i (in X) and e_d (in X') are different kinds of object; comparing or orthogonalizing them requires first mapping one into the other's space via the Riesz map, and then using *that* space's natural inner product. Using the raw Euclidean inner product throughout (as an early draft of this design did) happens to reproduce the correct answer for a single one-hot e_d – every diagonal-respecting inner product agrees on a coordinate-axis-aligned vector – but is not correct in general (e.g. a ring-neighbor indicator combining several points), and more importantly requires the fitting code to reach into M_2 (and its inverse) explicitly to do the orthogonalization correctly. That is exactly the kind of mass-matrix leakage this note is trying to eliminate.

3 Noise whitening

Define, per row (all quantities below implicitly depend on ρ through m_ρ and the batch):

$$\widehat{z}_\ell := M_2^{1/2} z_\ell, \tag{3}$$

$$\widehat{\phi}_i(\theta) := \sqrt{m_\rho}\; M_2^{1/2} \phi_i(\theta), \tag{4}$$

$$\widehat{e}_d := \sqrt{m_\rho}\; M_2^{-1/2} e_d, \tag{5}$$

$$\widehat{y}_\ell := y_\ell(\rho)/\sqrt{m_\rho}. \tag{6}$$

$M_2^{1/2}$ is diagonal, hence symmetric, so $(M_2\phi_i)\cdot z_\ell = (M_2^{1/2}\phi_i)\cdot(M_2^{1/2} z_\ell)$, giving $\widehat{\phi}_i \cdot \widehat{z}_\ell = \sqrt{m_\rho}\, g_\ell^{(i)}$; likewise $\widehat{e}_d \cdot \widehat{z}_\ell = \sqrt{m_\rho}\,(M_2^{-1/2} e_d)\cdot(M_2^{1/2} z_\ell) = \sqrt{m_\rho}\, e_d \cdot z_\ell = \sqrt{m_\rho}\, h_\ell^{(d)}$. Dividing (2) by $\sqrt{m_\rho}$:

$$\widehat{y}_\ell \;\approx\; \sum_i c_i \big(\widehat{\phi}_i(\theta)\cdot \widehat{z}_\ell\big) \;+\; \sum_d s_d \big(\widehat{e}_d \cdot \widehat{z}_\ell\big). \tag{7}$$

This is an ordinary (unweighted) linear regression in ℓ: every mass matrix has been absorbed into the four definitions (3)–(6), and (7) itself is mass-free.

Note on $\widehat{e}_d$: M_1 contributes the same $\sqrt{m_\rho}$ factor to *both* terms of (1) (there is one row mass, shared), so $\widehat{e}_d$ needs the same $\sqrt{m_\rho}$ prefactor as $\widehat{\phi}_i$ even though it does not involve M_2 (which enters with the opposite sign of exponent, $-1/2$ vs. $+1/2$, reflecting that e_d lives in X' where ϕ_i lives in X). An earlier pass at this derivation omitted the $\sqrt{m_\rho}$ on $\widehat{e}_d$; this was caught by checking (7) term by term against (2) before implementing it, and confirmed numerically (`test_whitening.cpp`) by constructing an explicit H from (1) and checking that the whitened quantities reproduce $y_\ell(\rho)$ to machine precision.

4 Consequences

Orthonormality is inherited directly. $\widehat{\phi}_i \cdot \widehat{\phi}_{i'} = m_\rho\,(M_2^{1/2}\phi_i)\cdot(M_2^{1/2}\phi_{i'}) = m_\rho\,\phi_i^\top M_2 \phi_{i'} \approx m_\rho\,\delta_{ii'}$: the whitened smooth basis vectors are, in the ordinary Euclidean sense, exactly as orthonormal (up to the constant m_ρ) as the M_2-orthonormality already established for the sampled LG modes. No separate weighted inner product is needed to see this – it falls out of using the plain dot product on already-whitened vectors.

Plain-Euclidean orthogonalization is now correct by construction. The session-2026-07-23 analysis found that projecting a smooth feature against an extra basis correctly requires the M_2^{-1}-weighted inner product on X': $\widehat{\phi}_p^\flat \mapsto \widehat{\phi}_p^\flat - e_i\,(e_i^\top M_2^{-1} e_i)^{-1}(e_i^\top M_2^{-1}\widehat{\phi}_p^\flat)$, $\widehat{\phi}_p^\flat := M_2\phi_p$. Checking this against the whitened quantities here: $\widehat{\phi}_p \cdot \widehat{e}_i = (M_2^{1/2}\phi_p)\cdot(M_2^{-1/2}e_i) = \phi_p \cdot e_i$ and $\widehat{e}_i \cdot \widehat{e}_i = e_i^\top M_2^{-1} e_i$ (the $\sqrt{m_\rho}$ factors cancel in the ratio) – identical projection coefficients. So plain-Euclidean orthogonalization of $\widehat{\phi}_p$ against $\widehat{e}_i$ *is* the earlier, carefully-derived M_2^{-1}-weighted projection; whitening does not introduce a different answer, it removes the need to ever justify or implement a non-Euclidean inner product near the fitting code.

Derivatives compose for free. $\sqrt{m_\rho}\,M_2^{1/2}$ and $\sqrt{m_\rho}\,M_2^{-1/2}$ are fixed, θ-independent, symmetric linear operators (diagonal matrices). Consequently:

- the JVP of $\widehat{\phi}_i$ is that operator applied to the JVP of the raw $\phi_i(\theta)$ – i.e. apply it to whatever `grad_eval_lg_nd` · `jvp_T` (chain rule through the pullback) already produces;
- because the operator is symmetric, the VJP is the *same* operator applied to the incoming cotangent *before* it is fed into the existing VJP chain.

This is the same "compose with a fixed linear map" pattern used already for Stage 1/Stage 2 composition in `ellipsoid_transform.hpp` and for the extra-basis projection in the 2026-07-23 design discussion. No new derivative formulas are needed anywhere; `lg_functions.hpp` and `ellipsoid_transform.hpp` are unchanged.

5 Resulting architecture

- **Mass-free composed feature layer** (`lg_ellipsoid_feature.hpp`): $\phi_i(x;\theta) := \psi_i(T(\theta,x))$ and its JVP/VJP, built purely by composing the two existing modules via the chain rule. Knows nothing about mass matrices.

- **Whitening interface layer** (`whitening.hpp`): the only place M_1, M_2 appear. Provides (a) `whiten_probes`, `whiten_data`, `whiten_extra` for the user-supplied raw probe/data/extra-basis arrays, and (b) whitened wrappers around the feature layer's eval/JVP/VJP, applying $\sqrt{m_\rho}\, M_2^{\pm 1/2}$ once.
- **Generic VarPro core** (not yet built): takes whitened probes, whitened data, and the whitened-basis eval/JVP/VJP as callables. Performs the actual per-row fit (orthogonalize the smooth basis against the extra basis, solve the inner linear least squares, form the reduced residual, provide its θ-Jacobian to the outer Levenberg–Marquardt loop) in ordinary Euclidean terms throughout. Never imports or references a mass matrix.